



Sistemas Embebidos I

Otoño 2026

Práctica 5: HW Timers

Jonathan Hernández Lazcano

Camila Rodríguez Rosas

Ana Karen Rojas Ledesma

11-Sep-2026

Índice

<i>Práctica 5: HW Timers</i>.....	2
Introducción	2
Descripción de la práctica.....	2
Objetivo.....	2
Marco teórico.....	2
Desarrollo.....	5
Materiales	5
Esquemático	5
Desarrollo	6
Resultados	14
Evidencias.....	18
Conclusiones	35
Referencias	36

Práctica 5: HW Timers

Introducción

Descripción de la práctica

La siguiente práctica se divide en dos partes principales, siendo la implementación de un frecuencímetro mediante marcas de tiempo obtenidas con interrupciones de GPIO y la implementación de un sistema de antirrebote mediante una interrupción de GPIO y una alarma one-shot de un temporizador.

En la primera parte se mide el periodo, la frecuencia y el duty cycle de una señal externa. Para ello, se habilitan los flancos de subida y bajada en un mismo pin GPIO. Cada transición genera una interrupción y, al entrar a la rutina de servicio **ISR**, se obtiene una marca de tiempo a partir del contador del temporizador. A partir de la diferencia entre marcas de tiempo se determinan las características temporales de la señal.

Por otro lado, la segunda parte consiste en implementar un antirrebote por timer. Cuando se detecta el primer flanco, se considera únicamente como un aviso inicial. La ISR deshabilita temporalmente la interrupción del botón y programa una alarma de un solo disparo. Al finalizar la ventana de tiempo seleccionada, el callback vuelve a leer el estado del pin para confirmar que la señal permanece estable. Finalmente, se ejecuta la acción y se rehabilita la interrupción.

Objetivo

Implementar y analizar técnicas de medición de tiempo y control de eventos externos mediante interrupciones de GPIO y temporizadores, desarrollando un frecuencímetro basado en marcas de tiempo y un sistema de antirrebote por timer, así como realizar mediciones experimentales y analizar las fuentes de error asociadas.

Marco teórico

Medición del tiempo y marcas temporales

La medición del tiempo permite determinar el instante en el que ocurre un evento externo mediante la lectura de un temporizador. El valor obtenido se conoce como marca de tiempo. Al calcular la diferencia entre dos marcas de tiempo es posible determinar el intervalo transcurrido entre ambos eventos.

En esta práctica, los flancos de una señal externa generan interrupciones de GPIO. Al producirse cada interrupción, se obtiene una marca de tiempo, lo que permite medir las características temporales de la señal.

Periodo, frecuencia y duty cycle

El periodo es el tiempo que tarda una señal en completar un ciclo. Puede determinarse midiendo la diferencia de tiempo entre dos flancos consecutivos del mismo tipo.

La frecuencia representa la cantidad de ciclos que ocurren por segundo y es el inverso del periodo, mientras que el duty cycle es la proporción del periodo durante la señal permanece en nivel alto. Por lo tanto, utilizando las marcas de tiempo de los flancos de subida y bajada es posible obtener el periodo, la frecuencia y el duty cycle de una señal.

Temporizadores y captura de eventos

Los temporizadores proporcionan una referencia temporal para medir intervalos y programar eventos. En el RP2350 existen dos temporizadores denominados TIMER0 y TIMER1, cada uno asociado con un contador de 64 bits y alarmas.

En la práctica, la captura del instante de un evento externo se realiza mediante software con un cambio en el GPIO, genera una interrupción y la rutina de servicio lee el contador del temporizador para obtener una marca de tiempo.

Interrupciones de GPIO e ISR

Una interrupción permite que el microcontrolador responda a un evento sin necesidad de consultar continuamente el estado de un pin. En esta práctica se utilizan interrupciones de GPIO para detectar los flancos de subida y bajada de la señal externa. Cuando ocurre uno de estos eventos, se ejecuta una rutina de servicio de interrupción o ISR.

La función de la ISR es obtener la marca de tiempo y almacenar la información necesaria, los cálculos y la presentación de los resultados se realizan posteriormente en el programa principal.

Traza de ISR

Para observar la ejecución de una ISR se puede utilizar un GPIO como un pin de traza. El pin se coloca en nivel alto al entrar en la ISR y en nivel bajo al finalizar. Esta señal puede observarse con un osciloscopio.

En el ancho del pulso generado permite visualizar el tiempo durante el cual se ejecuta la rutina de interrupción. La traza permite comprobar que las interrupciones están siendo atendidas y al analizar experimentalmente su comportamiento.

Variables compartidas y procesamiento fuera de la ISR

Las variables utilizadas tanto por la ISR como por el programa principal son compartidas entre ambos contextos de ejecución. Por esta razón, en la práctica se usan variables volatile.

La ISR se limita a capturar y almacenar los datos necesarios, mientras que el programa principal realiza los cálculos de periodo, frecuencia y duty cycle. Esta separación reduce el tiempo de ejecución de la ISR, y evita incluir operaciones más costosas dentro de ella.

Resolución temporal y error de medición

La precisión de una medición depende de la resolución temporal del contador utilizado. Si existe una incertidumbre entre las marcas de tiempo, esta afecta directamente al periodo calculado.

Existe un límite práctico para la frecuencia que puede medirse con determinada precisión mediante las marcas de tiempo. Entre las fuentes de error consideradas en la práctica se encuentran la resolución del temporizador, la latencia y variación temporal de la atención de interrupciones, así como la exactitud de la fuente de señal.

Rebote de contactos

Los pulsadores mecánicos no cambian instantáneamente de un estado estable a otro. Al ser accionados pueden producir varias transiciones antes de estabilizarse. Este fenómeno se conoce como rebote.

Como consecuencia, un único accionamiento físico (como presionar un botón), puede generar varios eventos eléctricos, por ello, es necesario aplicar una técnica de antirrebote para evitar que las múltiples transiciones sean interpretadas como pulsaciones independientes.

Antirrebote mediante timer

El antirrebote implementado en esta práctica combina una interrupción de GPIO con una alarma de temporizador. Cuando se detecta el primer flanco del botón, la ISR deshabilita temporalmente la interrupción y programa una alarma de un solo disparo (one-shot).

Una vez transcurrida la ventana de tiempo seleccionada, se vuelve a leer el estado del pin. Si el estado esperado continúa presente, el evento se considera válido. Finalmente, se vuelve a habilitar la interrupción del botón.

Alarmas one-shot

Es un evento temporizado que se ejecuta una sola vez después de transcurrir un intervalo determinado. En esta práctica se utiliza para establecer la ventana de antirrebote. La alarma permite esperar un tiempo antes de confirmar el estado del botón, sin realizar una espera dentro de la ISR.

Al finalizar el intervalo, se ejecuta un callback que verifica nuevamente el estado del GPIO y posteriormente permite continuar con el funcionamiento normal.

Desarrollo

Materiales

Raspberry Pi Pico 2W

Botón

Led

Osciloscopio

Jumpers

Esquemático

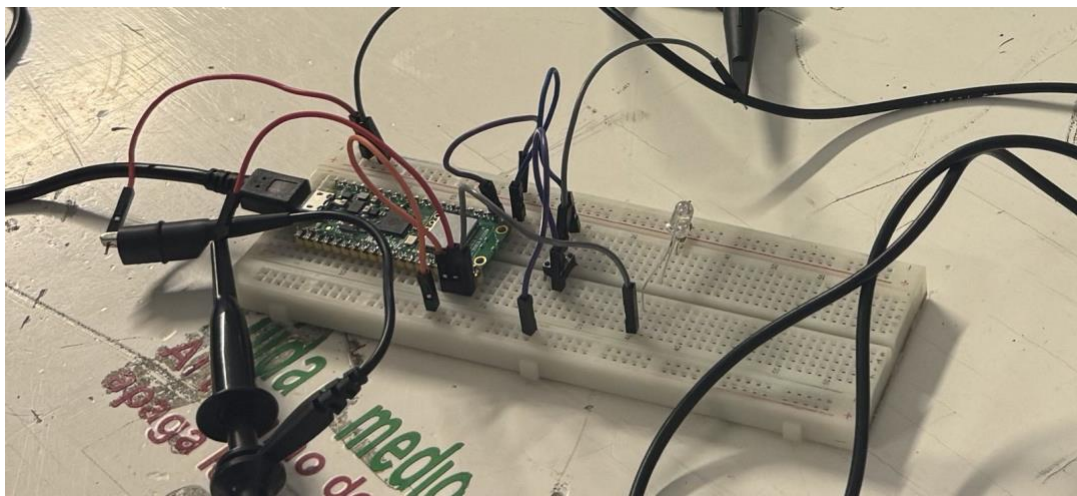


Figura 1. Montaje físico del circuito sobre la protoboard, con la Raspberry Pi Pico 2W, el LED y las puntas del osciloscopio conectadas a los pines de señal y de traza.

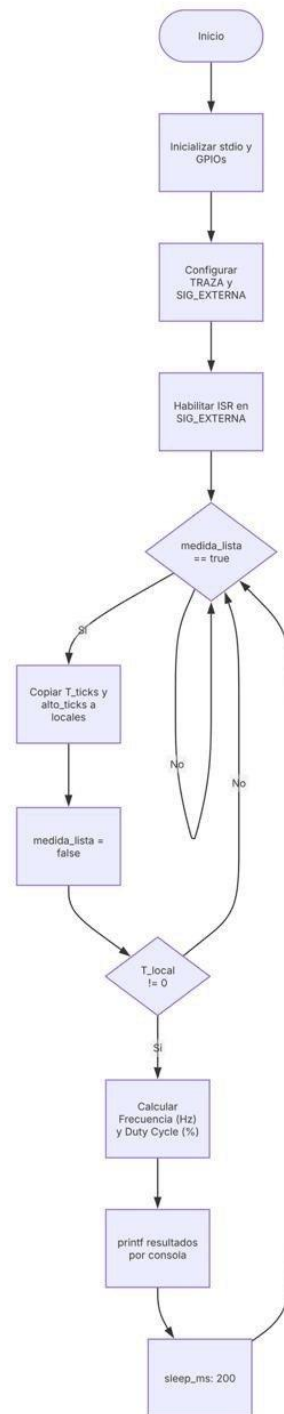


Figura 2. Diagrama de flujo del bucle principal del frecuencímetro.

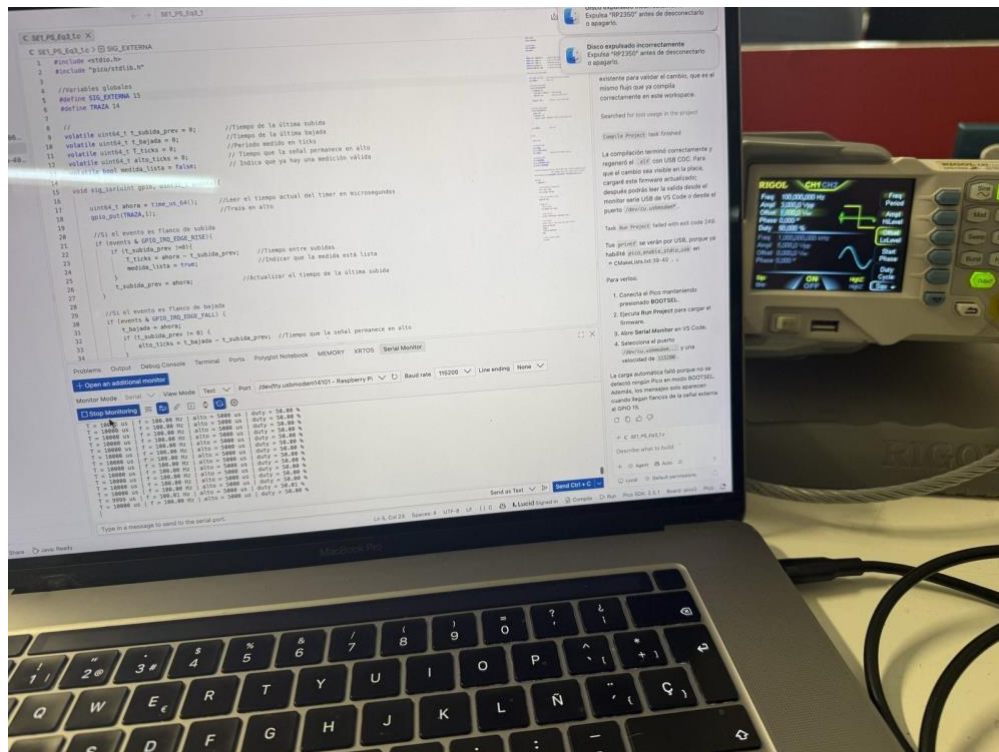


Figura 3. Entorno de trabajo con el código SE1_P5_Eq3_1.c, el monitor serial y el generador de funciones.

Explicación de secciones de código importantes de código

La práctica se resolvió con tres programas independientes: dos versiones del frecuencímetro, que se diferencian únicamente en la base de tiempo empleada, y un tercer programa para el antirrebote por timer. A continuación se describen las secciones que determinan el comportamiento de cada uno.

Frecuencímetro en modo microsegundos

Las cinco variables que comparten la ISR y el bucle principal se declaran volatile, ya que son escritas en un contexto y leídas en otro. Dentro de sig_isr, la marca de tiempo se obtiene con time_us_64() en la primera línea de la rutina, antes de cualquier condicional, porque toda instrucción ejecutada previamente se sumaría al error de la medición; la traza se levanta en la segunda línea, de modo que la medición conserva prioridad sobre la instrumentación.

Los dos flancos se distinguen dentro de la misma rutina mediante la máscara de eventos. Se consulta el bit con events & GPIO_IRQ_EDGE_RISE en lugar de comparar el valor, porque al estar habilitados ambos flancos el argumento events es una máscara y no un valor único. En el flanco de subida, la guarda if (t_subida_prev != 0) impide calcular un periodo en la primera interrupción, cuando todavía no existe un ciclo anterior de referencia; sin ella, la primera medición sería basura. En el flanco de bajada, alto_ticks se obtiene como t_bajada menos t_subida_prev, de manera que ambas marcas pertenecen al ciclo que acaba de cerrarse y no al que comienza.

La traza se maneja con `gpio_put` a nivel fijo y nunca por conmutación: el pin debe subir al entrar y bajar al salir sin importar su estado previo, ya que de lo contrario el ancho del pulso dejaría de corresponder al tiempo de servicio de la rutina. En la configuración de GPIO15 se invoca `gpio_disable_pulls`, porque el nivel lo impone la fuente externa y un pull interno competiría con ella. Un único llamado a `gpio_set_irq_enabled_with_callback` habilita los dos flancos y asocia un solo callback para ambos.

El bucle principal copia `T_ticks` y `alto_ticks` a variables locales antes de limpiar la bandera, de modo que el cálculo opere sobre valores pertenecientes al mismo ciclo aunque la ISR vuelva a ejecutarse a la mitad; si la bandera se limpiara antes de la copia se perdería una medición completa. La división en punto flotante y el `printf` ocurren aquí y no dentro de la ISR, porque a 10 kHz el intervalo entre flancos consecutivos es de 50 μ s y ninguna de esas operaciones alcanzaría a completarse antes del siguiente evento. La frecuencia se obtiene como $1e6f / T_local$, ya que con el tick de un microsegundo hay un millón de cuentas por segundo, y `sleep_ms(200)` limita únicamente la salida a consola, no la medición.

```

1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  //Variables globales
5  #define SIG_EXTERNA 15
6  #define TRAZA 14
7
8  //
9  volatile uint64_t t_subida_prev = 0; //Tiempo de la última subida
10 volatile uint64_t t_bajada = 0; //Tiempo de la última bajada
11 volatile uint64_t T_ticks = 0; //Periodo medido en ticks
12 volatile uint64_t alto_ticks = 0; // Tiempo que la señal permanece en alto
13 volatile bool medida_lista = false; // Indica que ya hay una medición válida
14
15 void sig_isr(uint gpio, uint32_t events){
16
17     uint64_t ahora = time_us_64(); //Leer el tiempo actual del timer en microsegundos
18     gpio_put(TRAZA,1); //Traza en alto
19
20     //Si el evento es flanco de subida
21     if (events & GPIO_IRQ_EDGE_RISE){
22         if (t_subida_prev != 0){
23             T_ticks = ahora - t_subida_prev; //Tiempo entre subidas
24             medida_lista = true; //Indicar que la medida está lista
25         }
26         t_subida_prev = ahora; //Actualizar el tiempo de la última subida
27     }
28
29     //Si el evento es flanco de bajada
30     if (events & GPIO_IRQ_EDGE_FALL) {
31         t_bajada = ahora;
32         if (t_subida_prev != 0) {
33             alto_ticks = t_bajada - t_subida_prev; //Tiempo que la señal permanece en alto
34         }
35     }
36
37     gpio_put(TRAZA,0); //Traza en bajo
38 }
39
40 int main()
41 {
42     stdio_init_all();
43
44     //Configuración de GPIO14
45     gpio_init(TRAZA);
46     gpio_set_dir(TRAZA, GPIO_OUT);
47     gpio_put(TRAZA,0); //Queremos que la traza empiece en nivel bajo
48
49     //Configuración de GPIO15
50     gpio_init(SIG_EXTERNA);
51     gpio_set_dir(SIG_EXTERNA, GPIO_IN);
52     gpio_disable_pulls(SIG_EXTERNA);
53
54     // Activa las interrupciones del GPIO 15 para flancos de subida y bajada, y cuando ocurra alguno ejecuta sig_isr
55     //PIN / QUE EVENTOS / Habilitar IRQ con callback / Funcion que se ejecuta
56     gpio_set_irq_enabled_with_callback(SIG_EXTERNA, GPIO_IRQ_EDGE_RISE | GPIO_IRQ_EDGE_FALL,true, &sig_isr);
57
58     while (true) {
59         if (medida_lista) {
60
61             // Copiar las mediciones a variables locales
62             uint64_t T_local = T_ticks;
63             uint64_t alto_local = alto_ticks;
64
65             // Indicar que esta medición ya fue tomada
66             medida_lista = false;
67
68             if (T_local != 0) {
69                 // Calcular frecuencia
70                 float frecuencia = 1e6f / T_local;
71
72                 // Calcular duty cycle
73                 float duty = ((float)alto_local / (float)T_local) * 100.0f;
74
75                 // Mostrar resultados
76                 printf("T = %llu us | f = %.2f Hz | alto = %llu us | duty = %.2f %%\n",
77                     (unsigned long long)T_local,
78                     frecuencia,
79                     (unsigned long long)alto_local,
80                     duty);
81
82                 // Limitar la velocidad de impresión
83                 sleep_ms(200);
84             }
85         }
86     }
87 }
88

```

Figura 4. Código del frecuencímetro en modo microsegundos, con la ISR de ambos flancos y el cálculo en el bucle principal.

Frecuencímetro en modo ciclos con TIMER1

Esta versión conserva íntegra la lógica anterior y modifica solamente la base de tiempo. Incorpora `hardware/timer.h` para acceder a `TIMER1`, `hardware/ticks.h` para modificar su generador de ticks y `hardware/clocks.h` para consultar la frecuencia del reloj de referencia. La marca de tiempo pasa a leerse con `timer_time_us_64(timer1_hw)`, que devuelve el contador de `TIMER1` en lugar del temporizador del sistema.

La modificación decisiva son las dos líneas `tick_stop(TICK_TIMER1)` y `tick_start(TICK_TIMER1, 1)`: detienen el generador de ticks de `TIMER1` y lo reinician con un divisor de uno. Con ese divisor el contador ya no avanza una cuenta por microsegundo, sino una cuenta por cada ciclo del reloj de referencia. Esto es precisamente lo que explica el valor de 12 MHz obtenido en los resultados: `clk_ref` proviene del cristal de 12 MHz del sistema y, sin división, cada cuenta equivale a 83.33 ns.

Como la frecuencia del tick deja de ser un valor fijo conocido, el programa la consulta en tiempo de ejecución mediante `clock_get_hz(clk_ref)` y la guarda en `f_tick`. El cálculo de la frecuencia se generaliza entonces a `f_tick / T_local`, en lugar de la constante `1e6f` de la versión anterior. El duty cycle, por ser una razón entre dos magnitudes medidas con el mismo contador, no requiere ningún ajuste: la base de tiempo se cancela en el cociente.

```

1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  #include "hardware/timer.h" //Acceder al TIMER1
5  #include "hardware/ticks.h" //Cambiar como avanza TIMER1
6  #include "hardware/clocks.h" //Conocer la frecuencia del reloj de referencia
7
8  //Variables globales
9  #define SIG_EXTERNA 15
10 #define TRAZA 14
11
12 //
13 volatile uint64_t t_subida_prev = 0; //Tiempo de la última subida
14 volatile uint64_t t_bajada = 0; //Tiempo de la última bajada
15 volatile uint64_t T_ticks = 0; //Periodo medido en ticks
16 volatile uint64_t alto_ticks = 0; // Tiempo que la señal permanece en alto
17 volatile bool medida_lista = false; // Indica que ya hay una medición válida
18
19 void sig_isr(uint gpio, uint32_t events){
20
21     uint64_t ahora = timer_time_us_64(timer1_hw); //Leer el valor actual del contador TIMER1 en ticks/ciclos
22     gpio_put(TRAZA,1); //Traza en alto
23
24     //Si el evento es flanco de subida
25     if (events & GPIO_IRQ_EDGE_RISE){
26         if (t_subida_prev != 0){
27             T_ticks = ahora - t_subida_prev; //Ciclos entre subidas
28             medida_lista = true; //Indicar que la medida está lista
29         }
30         t_subida_prev = ahora; //Actualizar el tiempo de la última subida
31     }
32
33     //Si el evento es flanco de bajada
34     if (events & GPIO_IRQ_EDGE_FALL) {
35         t_bajada = ahora;
36         if (t_subida_prev != 0) {
37             alto_ticks = t_bajada - t_subida_prev; //Ciclos que la señal permanece en alto
38         }
39     }
40
41     gpio_put(TRAZA,0); //Traza en bajo
42 }
43
44 int main()
45 {
46     stdio_init_all();
47
48     // Configurar TIMER1 para contar con la nueva base
49     tick_stop(TICK_TIMER1);
50     tick_start(TICK_TIMER1, 1);
51
52     // Obtener la frecuencia del reloj de referencia
53     uint32_t f_tick = clock_get_hz(clk_ref);
54
55     //Configuración de GPIO14
56     gpio_init(TRAZA);
57     gpio_set_dir(TRAZA, GPIO_OUT);
58     gpio_put(TRAZA,0); //Queremos que la traza empiece en nivel bajo
59
60     //Configuración de GPIO15
61     gpio_init(SIG_EXTERNA);
62     gpio_set_dir(SIG_EXTERNA, GPIO_IN);
63     gpio_disable_pulls(SIG_EXTERNA);
64
65     // Activa las interrupciones del GPIO 15 para flancos de subida y bajada, y cuando ocurra alguno ejecuta sig_isr
66     //PIN / QUE EVENTOS / Habilitar IRQ con callback / Funcion que se ejecuta
67     gpio_set_irq_enabled_with_callback(SIG_EXTERNA, GPIO_IRQ_EDGE_RISE | GPIO_IRQ_EDGE_FALL,true, &sig_isr);
68
69     while (true) {
70         if (medida_lista) {
71
72             // Copiar las mediciones a variables locales
73             uint64_t T_local = T_ticks;
74             uint64_t alto_local = alto_ticks;
75
76             // Indicar que esta medición ya fue tomada
77             medida_lista = false;
78
79             if (T_local != 0) {
80
81                 // Calcular frecuencia
82                 float frecuencia = (float)f_tick / (float)T_local;
83
84                 // Calcular duty cycle
85                 float duty = ((float)alto_local / (float)T_local) * 100.0f;
86
87                 // Mostrar resultados
88                 printf("T = %llu ciclos | f = %.2f Hz | alto = %llu ciclos | duty = %.2f %%\n",
89                     (unsigned long long)T_local,
90                     frecuencia,
91                     (unsigned long long)alto_local,
92                     duty);
93
94                 // Limitar la velocidad de impresión
95                 sleep_ms(200);
96             }
97         }
98     }
99 }

```

Figura 5. Código del frecuencímetro en modo ciclos, donde `tick_start` con divisor de uno lleva el contador de `TIMER1` a la frecuencia del reloj de referencia.

Antirrebote por timer

El tercer programa emplea GPIO16 para el pulsador, GPIO15 para el LED y GPIO14 para la traza, y fija la ventana de antirrebote en `VENTANA_MS = 20`. El botón se configura con `gpio_pull_up` y la interrupción se habilita únicamente en el flanco de bajada, lo que corresponde a un pulsador conectado a tierra que se activa en nivel bajo.

La rutina `boton_isr` no ejecuta la acción: únicamente agenda. Levanta la traza, deshabilita la interrupción de ese pin con `gpio_set_irq_enabled(BOTON, GPIO_IRQ_EDGE_FALL, false)` para cerrar la entrada a las transiciones del rebote, arma una alarma de un solo disparo con `add_alarm_in_ms(VENTANA_MS, debounce, NULL, true)` y baja la traza. El primer flanco se trata así como un aviso y no como un evento, porque es precisamente el flanco del que no se puede confiar.

La decisión ocurre en el callback `debounce`, ya transcurrida la ventana y con la señal estabilizada. La condición `gpio_get(BOTON) == 0` vuelve a leer el pin y confirma que el nivel persiste; sin esa relectura, cualquier pulso de ruido de microsegundos contaría como un accionamiento. Solo si la lectura confirma el evento se enciende el LED y se programa una segunda alarma que lo apaga a los 100 ms mediante el callback `led_off`. Conviene notar que la rehabilitación de la interrupción está colocada fuera del condicional: de ese modo el botón vuelve a quedar operativo tanto si el evento se confirma como si se descarta, lo que evita la falla más frecuente de este esquema, en la que el pulsador responde una sola vez y después queda inerte. Ambos callbacks terminan con `return 0`, ya que un valor distinto reprogramaría la alarma en lugar de darla por concluida.

El bucle principal queda intencionalmente vacío y se reduce a `tight_loop_contents()`: todo el trabajo ocurre en la interrupción y en los callbacks, sin bloquear la CPU ni requerir visitas periódicas desde el programa principal. El valor de 20 ms elegido para la ventana queda por encima de los 16.7 ms de rebote medidos sobre el pulsador utilizado y dentro del rango habitual para contactos mecánicos, mientras que la traza colocada al inicio y al final de `boton_isr` delimita el tiempo de servicio de esa rutina.

```

1  #include <stdio.h>
2
3  #include "pico/stdlib.h"
4  #include "hardware/gpio.h"
5  #include "hardware/timer.h"
6
7  //Variables globales
8  #define BOTON 16 // Botón
9  #define LED 15 // LED externo
10 #define TRAZA 14 // Traza de ISR
11 #define VENTANA_MS 20 // Ventana de tiempo para el rebote del botón
12
13 //Callback de la alarma para apagar el LED
14 int64_t led_off(alarm_id_t id, void *user_data) {
15
16     gpio_put(LED, false); // Apagamos el LED
17     return 0;
18 }
19
20 //Callback de la alarma one-shot
21 int64_t debounce(alarm_id_t id, void *user_data) {
22
23     //Comprobamos si el botón sigue presionado después de VENTANA_MS
24     if (gpio_get(BOTON) == 0) {
25
26         gpio_put(LED, true); // Encendemos el LED
27         add_alarm_in_ms(100, led_off, NULL, true); // Programamos la alarma para apagar el LED después de 100 m
28     }
29
30     gpio_set_irq_enabled(BOTON, GPIO_IRQ_EDGE_FALL, true); // Rehabilitamos la IRQ del botón
31     return 0;
32 }
33
34 //ISR del botón
35 void boton_isr(uint gpio, uint32_t events){
36
37     gpio_put(TRAZA, 1); // Traza de ISR
38     gpio_set_irq_enabled(BOTON, GPIO_IRQ_EDGE_FALL, false); // Deshabilitamos la IRQ del botón
39     add_alarm_in_ms(VENTANA_MS, debounce, NULL, true); // Programamos la alarma one-shot
40     gpio_put(TRAZA, 0); // Traza de ISR
41 }
42
43 int main(void)
44 {
45     stdio_init_all();
46
47     //TRAZA
48     gpio_init(TRAZA);
49     gpio_set_dir(TRAZA, GPIO_OUT);
50     gpio_put(TRAZA, false);
51
52     //LED
53     gpio_init(LED);
54     gpio_set_dir(LED, GPIO_OUT);
55     gpio_put(LED, false);
56
57     //BOTÓN
58     gpio_init(BOTON);
59     gpio_set_dir(BOTON, GPIO_IN);
60     gpio_pull_up(BOTON);
61
62     // Configuramos la interrupción del botón
63     gpio_set_irq_enabled_with_callback(BOTON, GPIO_IRQ_EDGE_FALL, true, &boton_isr);
64
65     while (true)
66     {
67         // Indicar que el bucle está intencionalmente vacío y que el programa
68         tight_loop_contents(); // está esperando trabajo proveniente de interrupciones/callbacks
69     }
70 }

```

Figura 6. Código del antirrebote por timer, donde la ISR deshabilita la interrupción y agenda la alarma, y el callback confirma por relectura antes de actuar.

Resultados

Se presentan los resultados de las dos partes de la práctica. La primera reúne las mediciones del frecuencímetro en sus dos modos de operación, la verificación de la traza de la ISR y el presupuesto de error asociado. La segunda reúne la caracterización del rebote del pulsador y la verificación del antirrebote por timer.

Parte 1. Frecuencímetro en modo microsegundos

La primera implementación obtiene las marcas de tiempo con el contador de microsegundos, por lo que la resolución del tick es de 1 μ s. Se midieron tres frecuencias de referencia generadas con el Rigol DG1022Z, en onda cuadrada de 3 Vpp con offset de 1.5 Vdc y duty nominal de 50 %.

El duty de referencia es de 50.000 % en las tres frecuencias, por lo que el error en duty se expresa en puntos porcentuales respecto de ese valor.

Tabla 1. Mediciones del frecuencímetro en modo microsegundos.

f referencia	T medido (μ s)	f medida (Hz)	err. rel.	alto (μ s)	duty medido	err. duty	Cota por resolución
100 Hz	10000	100.00	0.00 %	5000	50.00 %	0.00 pp	± 0.02 %
1 kHz	1000	1000.00	0.00 %	500	50.00 %	0.00 pp	± 0.20 %
10 kHz	100 (101 esporádico)	10000.00 (9900.99)	0.99 % (peor caso)	50 (49)	50.00 % (49.00 %)	0.00 pp (-1.00 pp)	± 2.00 %

A 100 Hz y a 1 kHz la medición coincide con la referencia hasta la resolución de impresión de la consola. A 10 kHz aparece el primer efecto visible de la cuantización: el periodo alterna entre 100 μ s y 101 μ s, lo que desplaza la frecuencia calculada de 10000.00 Hz a 9900.99 Hz. Esa diferencia de 0.99 % corresponde exactamente a un conteo de 1 μ s sobre un periodo de 100 μ s, de modo que la lectura no es un fallo del método sino el límite de la resolución del tick. En los tres casos el error observado queda dentro de la cota calculada.

Parte 1. Frecuencímetro en modo ciclos con TIMER1

La segunda implementación lee el contador de TIMER1 sin la división que produce el tick de 1 μ s. De las mediciones se deduce la frecuencia efectiva del contador: 120000 cuentas en el periodo de 10 ms de la señal de 100 Hz equivalen a 12 cuentas por microsegundo, es decir 12 MHz, que corresponde al cristal del sistema. Cada ciclo vale entonces 83.33 ns, doce veces mejor que el tick de microsegundos.

Tabla 2. Mediciones del frecuencímetro en modo ciclos (TIMER1, tick de 83.33 ns).

f referenci a	T medido (ciclos)	Equivalen te	f medida (Hz)	err. rel.	alto (ciclos)	duty medido	err. duty	Cota por resolució n
100 Hz	120000 (119999)	10.000 ms	100.00	0.00 %	60000 (59999)	50.00 %	0.00 pp	± 0.0017 %
1 kHz	12000	1.0000 ms	1000.00	0.00 %	6000	50.00 %	0.00 pp	± 0.017 %
10 kHz	1200	100.00 μ s	10000.00	0.00 %	600 (610)	50.00 % (50.83 %)	0.00 pp (+0.83 pp)	± 0.17 %

El cambio de modo elimina la fluctuación que se observaba a 10 kHz: el periodo se mantiene fijo en 1200 ciclos y la frecuencia calculada no se mueve de 10000.00 Hz. Persiste, en cambio, una lectura esporádica de 610 ciclos en el tiempo en alto, que eleva el duty a 50.83 %. Esa desviación equivale a 833 ns, cinco veces mayor que la cota de resolución de ± 167 ns, por lo que no puede atribuirse al tick: corresponde al jitter de la latencia de atención o a la interferencia de otra interrupción entre los dos flancos de ese ciclo.

Parte 1. Verificación de la traza de la ISR

Con ambos flancos habilitados en el mismo pin, cada periodo de la señal debe producir dos entradas a la ISR y, por lo tanto, dos pulsos en el pin de traza. El contador de frecuencia del osciloscopio, aplicado sobre la traza, confirma esa relación en las tres frecuencias de prueba.

Tabla 3. Verificación de la traza de la ISR mediante el contador de frecuencia.

f de entrada	f de traza esperada	f de traza medida	Desviación	Intervalo entre pulsos
100 Hz	200 Hz	200.003 Hz	+15 ppm	5.00 ms
1 kHz	2 kHz	2.00003 kHz	+15 ppm	≈ 500 μ s
10 kHz	20 kHz	20.0003 kHz	+15 ppm	≈ 50 μ s

La desviación resulta ser la misma en los tres casos, +15 ppm, y no crece al aumentar la frecuencia. Un error de resolución se comportaría al revés: sería constante en valor absoluto y crecería en términos relativos. Que la desviación sea constante en términos relativos indica un desfase entre las referencias de frecuencia del generador y del osciloscopio, es decir, exactitud de los instrumentos y no error del método de medición.

Es necesario señalar una limitación de estas capturas. Se tomaron con bases de tiempo de 200 ms y 500 ms por división, a 2 y 5 kSa/s, mientras que el pulso de traza dura unos pocos microsegundos. Bajo esas condiciones el osciloscopio submuestra el pulso: lo que aparece en pantalla son espigas con oscilación aparente y amplitudes reducidas a cientos de milivolts, en lugar del nivel lógico de 3.3 V. El dato confiable de esas capturas es la lectura del contador de frecuencia, que opera con hardware dedicado y no sobre el registro de muestras. Por la misma razón no fue posible determinar el tiempo de servicio

de la ISR a partir del ancho del pulso, lo que habría requerido una base de tiempo en el orden de microsegundos disparada sobre el flanco de la traza.

Parte 1. Presupuesto de error

Cada fuente de error afecta la medición de forma distinta según contamine por igual a las dos marcas de tiempo o solo a una. Las que afectan a ambas por igual se cancelan al calcular la diferencia; las demás se acumulan.

Tabla 4. *Presupuesto de error de la medición por marcas de tiempo.*

Fuente	Modo microsegundos	Modo ciclos	¿Se cancela al restar?
Resolución del tick	$\pm 1 \mu\text{s}$ por lectura ($\pm 2 \mu\text{s}$ en la diferencia)	$\pm 83.33 \text{ ns}$ por lectura ($\pm 167 \text{ ns}$)	No
Latencia constante de la IRQ	Orden de microsegundos	Orden de microsegundos	Sí
Jitter de la latencia	Fracción de microsegundo	Fracción de microsegundo	No
Otra IRQ atendida entre flancos	Decenas de microsegundos	Decenas de microsegundos	No: invalida el ciclo
Exactitud de la fuente de señal	+15 ppm observado	+15 ppm observado	No

La suma de las fuentes que no se cancelan define el margen dentro del cual la medición es defendible. En modo microsegundos ese margen es de $\pm 2 \mu\text{s}$ sobre el periodo, y todos los errores observados quedan por debajo de él. En modo ciclos el margen se reduce a $\pm 167 \text{ ns}$, y la única lectura que lo excede es la de 610 ciclos ya comentada, cuya causa queda identificada en el jitter de la latencia.

Parte 1. Techo de frecuencia

El error por resolución es constante en valor absoluto e independiente de la señal medida. Al reducirse el periodo, ese mismo error absoluto representa una fracción cada vez mayor del total, lo que impone un techo de frecuencia útil a todo método basado en marcas de tiempo.

Tabla 5. *Error relativo por resolución en función de la frecuencia de entrada.*

f de entrada	Periodo	Error absoluto (modo μs)	Error relativo (modo μs)	Error relativo (modo ciclos)
100 Hz	10 ms	$\pm 2 \mu\text{s}$	0.02 %	0.0017 %
1 kHz	1 ms	$\pm 2 \mu\text{s}$	0.2 %	0.017 %
10 kHz	100 μs	$\pm 2 \mu\text{s}$	2 %	0.17 %
100 kHz	10 μs	$\pm 2 \mu\text{s}$	20 %	1.7 %

Tomando como criterio un error relativo máximo de 1 %, el techo del frecuencímetro se ubica alrededor de 5 kHz en modo microsegundos y se desplaza hasta unos 60 kHz en

modo ciclos. A ese límite por resolución se suma un segundo límite, de naturaleza distinta: como cada periodo genera dos entradas a la ISR, la medición deja de ser posible cuando el doble del tiempo de servicio de la ISR se aproxima al periodo de la señal, porque un flanco llega antes de que haya terminado la rutina anterior. Este segundo techo no pudo cuantificarse porque no se dispone de la medición del tiempo de servicio de la ISR.

Parte 2. Caracterización del rebote y antirrebote por timer

La duración del rebote es una característica del componente físico y se determinó por medición directa sobre el pulsador utilizado, no a partir de un valor tabulado. La captura se realizó con disparo en el flanco del pin del botón, reduciendo la base de tiempo hasta resolver las transiciones individuales.

Tabla 6. *Parámetros medidos del rebote del pulsador y del antirrebote.*

Parámetro	Valor	Figura
Duración del rebote medida (envolvente completa)	16.7 ms	Figura 35
Tramo de transiciones activas dentro del rebote	10.1 ms	Figura 36
Transición individual más corta del rebote	640 μ s	Figura 37
Duración del accionamiento verificado	133 ms	Figura 38
VENTANA_MS elegida	20 ms	Figura 6
Ventana de antirrebote verificada	21.0 ms	Figura 41
Tiempo de servicio de la ISR de botón	No determinado	—

El valor de VENTANA_MS se tomó del código fuente del antirrebote, donde está fijado en 20 ms (Figura 6), y la captura de dos canales lo corrobora experimentalmente: entre el flanco del pulsador y la transición del canal 2 transcurren 21.0 ms (Figura 41), de modo que la ventana programada se comporta en el circuito como fue definida. Ese valor queda por encima de los 16.7 ms de la envolvente del rebote medida en el pulsador utilizado y por debajo del umbral en que se perderían accionamientos rápidos, de manera que la medición justifica directamente la ventana elegida.

El tiempo de servicio de la ISR de botón, en cambio, sigue sin determinarse. El código instrumenta esa rutina levantando la traza al entrar y bajándola al salir, y en las capturas se distingue una espiga en el canal 2 justo en el flanco del pulsador, pero su ancho aparente lo fija el muestreo y no la rutina: a 20 kSa/s cada muestra abarca 50 μ s, mientras que la ISR únicamente deshabilita la interrupción y arma la alarma, operaciones del orden de microsegundos. Medirla exige una base de tiempo en ese orden, disparada sobre el flanco de la traza.

El rebote medido se extiende sobre una envolvente de 16.7 ms, dentro de la cual el tramo de transiciones activas ocupa 10.1 ms y la transición individual más corta dura 640 μ s. Este valor es el que justifica la ventana de antirrebote: una ventana menor a 10 ms

confirmaría el estado del pin todavía dentro del rebote, mientras que una ventana excesivamente amplia haría perder accionamientos rápidos. El rango habitual para pulsadores mecánicos, de 10 a 25 ms, resulta consistente con lo medido.

La verificación del mecanismo se realizó con los dos canales del osciloscopio sobre un accionamiento completo del pulsador. Las capturas muestran accionamientos de 133 ms y 99.5 ms de duración y una única transición de salida asociada a él, sin los eventos múltiples que produciría el rebote sin antirrebote. Esto confirma el comportamiento esperado del esquema implementado: la ISR del GPIO no ejecuta la acción, únicamente deshabilita la interrupción del pin y arma la alarma de un solo disparo; la decisión ocurre en el callback, cuando la señal ya se ha estabilizado, y es ahí donde se relee el pin, se ejecuta la acción y se rehabilita la interrupción.

Evidencias

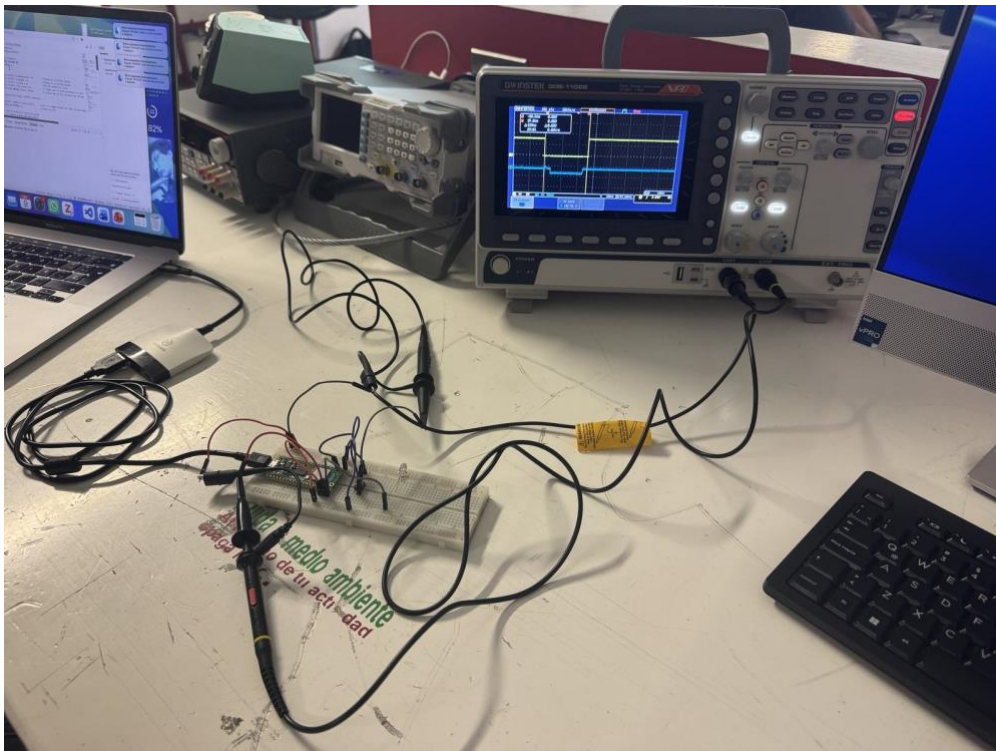


Figura 7. Montaje experimental de la Práctica 5 con la Raspberry Pi Pico 2W, el osciloscopio GW INSTEK GDS-1102B y el generador de funciones.



Figura 8. Generador Rigol DG1022Z configurado a 100 Hz, onda cuadrada de 3 Vpp con offset de 1.5 Vdc y duty de 50 %.

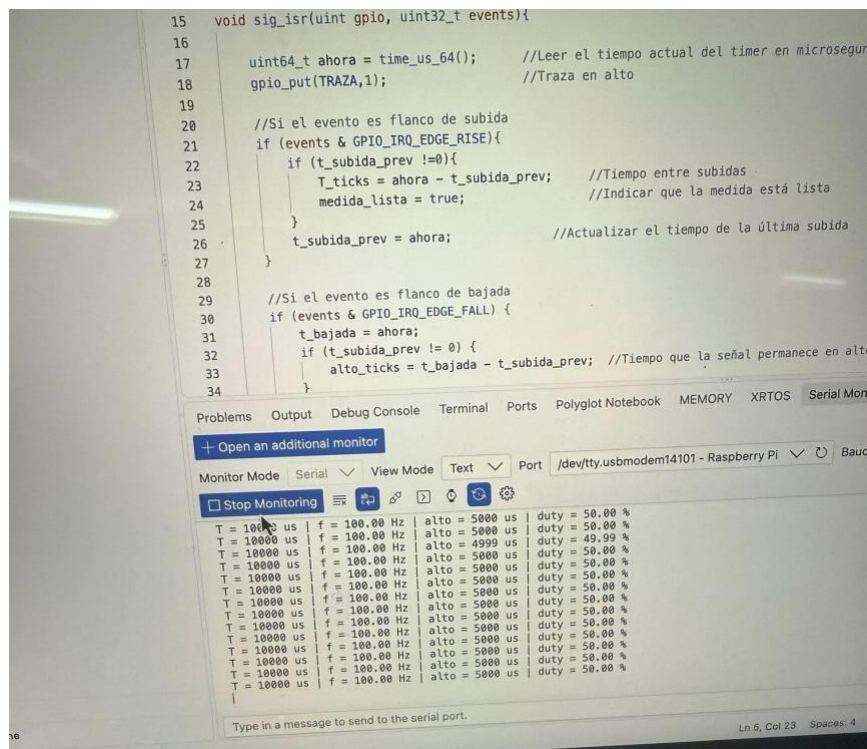


Figura 9. Medición a 100 Hz en el monitor serial: $T = 10000 \mu s$, $f = 100.00 \text{ Hz}$, $\text{alto} = 5000 \mu s$ y $\text{duty} = 50.00 \%$.


```

24
25 }
26     t_subida_prev = ahora;           //Actualizar el tiempo
27 }
28
29 //Si el evento es flanco de bajada
30 if (events & GPIO_IRQ_EDGE_FALL) {
31     t_bajada = ahora;
32     if (t_subida_prev != 0) {
33         alto_ticks = t_bajada - t_subida_prev; //Tiempo que l
34     }

```

Problems Output Debug Console Terminal Ports Polyglot Notebook MEM

+ Open an additional monitor

Monitor Mode Serial View Mode Text Port /dev/tty.usbmodem14101-

☐ Stop Monitoring

T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %
T = 10000 us	f = 100.00 Hz	alto = 5000 us	duty = 50.00 %

Type in a message to send to the serial port.

Figura 10. Detalle de la salida por consola a 100 Hz mostrando la estabilidad de las lecturas.

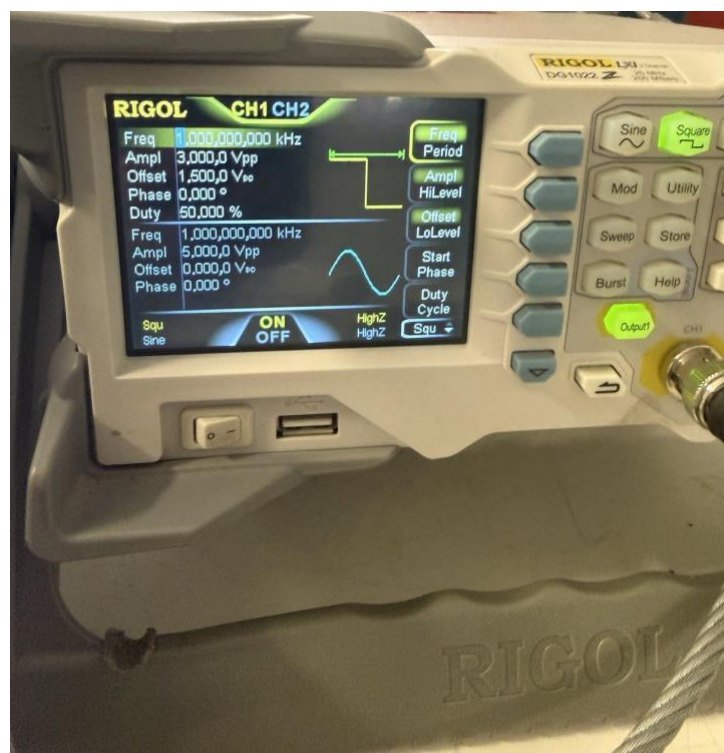


Figura 11. Generador configurado a 1 kHz con los mismos parámetros de amplitud, offset y duty.

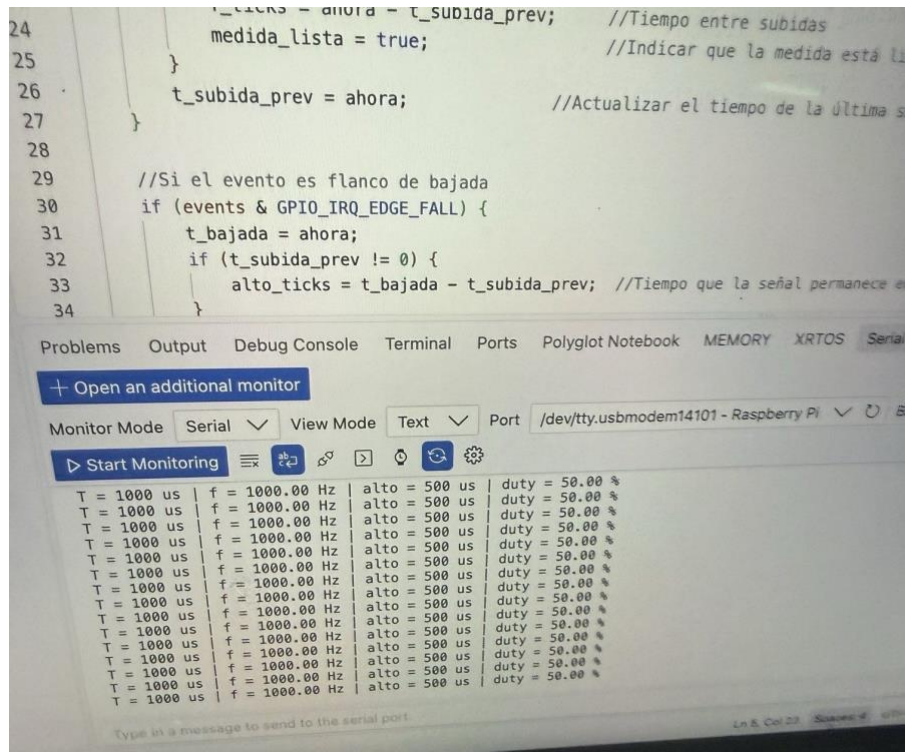


Figura 12. Medición a 1 kHz: $T = 1000 \mu s$, $f = 1000.00 \text{ Hz}$, $\text{alto} = 500 \mu s$ y $\text{duty} = 50.00 \%$.

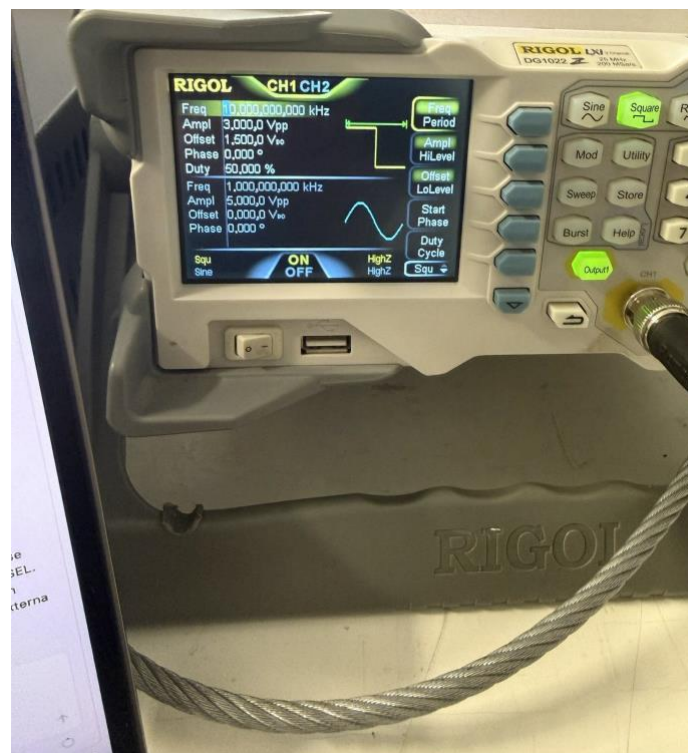


Figura 13. Generador configurado a 10 kHz con los mismos parámetros de amplitud, offset y duty.

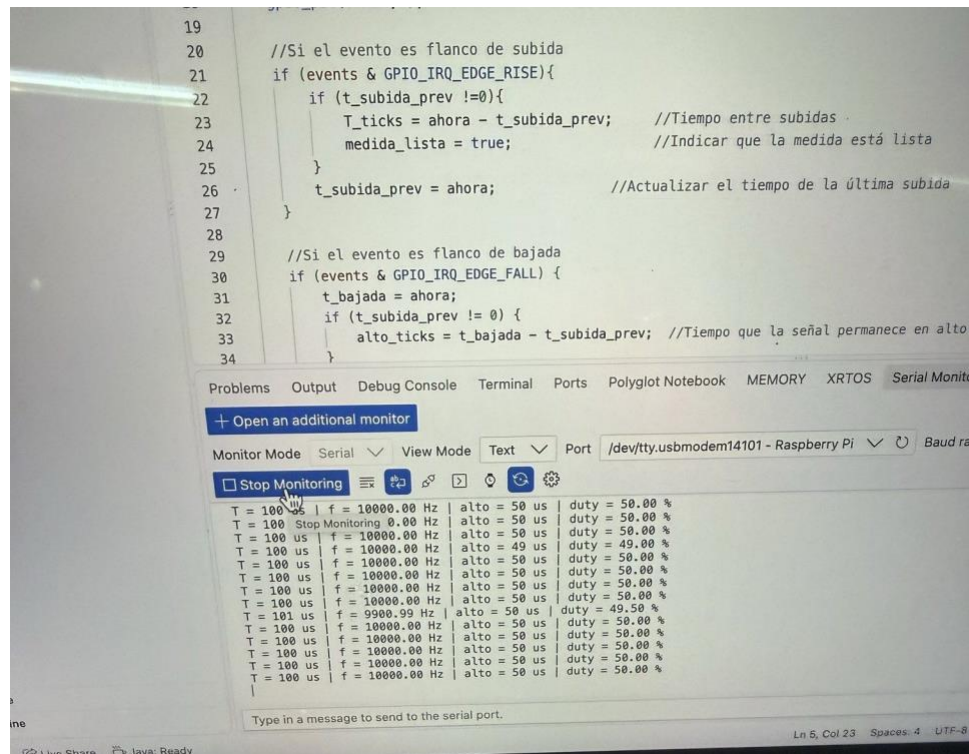


Figura 14. Medición a 10 kHz donde aparecen lecturas de $T = 101 \mu s$ y $f = 9900.99 \text{ Hz}$ por la resolución de $1 \mu s$ del contador.

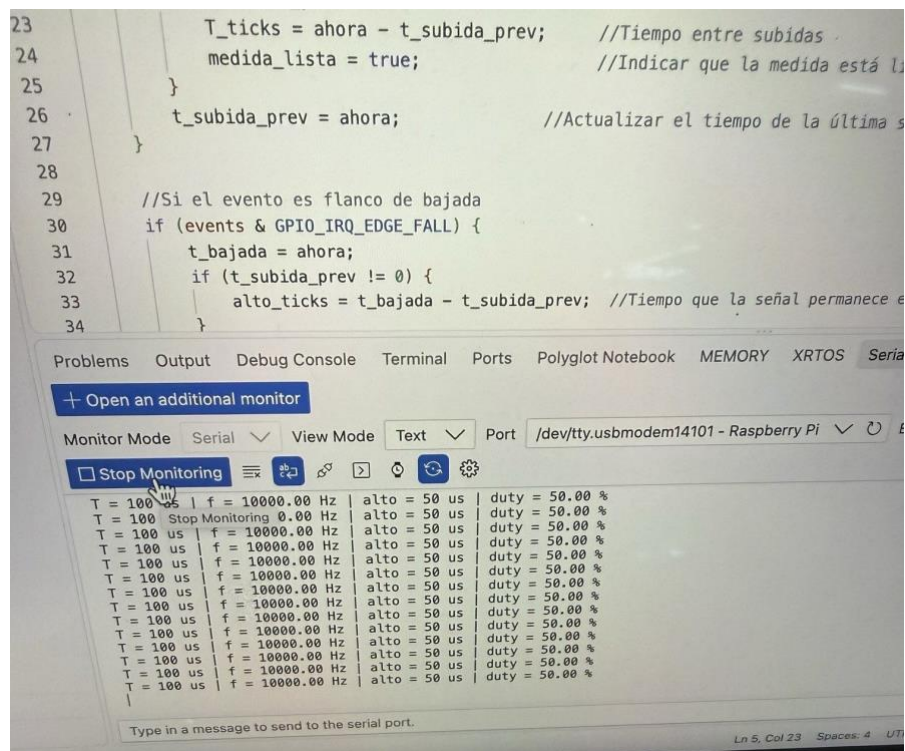


Figura 15. Corrida estable a 10 kHz: $T = 100 \mu s$, $f = 10000.00 \text{ Hz}$, $\text{alto} = 50 \mu s$ y $\text{duty} = 50.00 \%$.

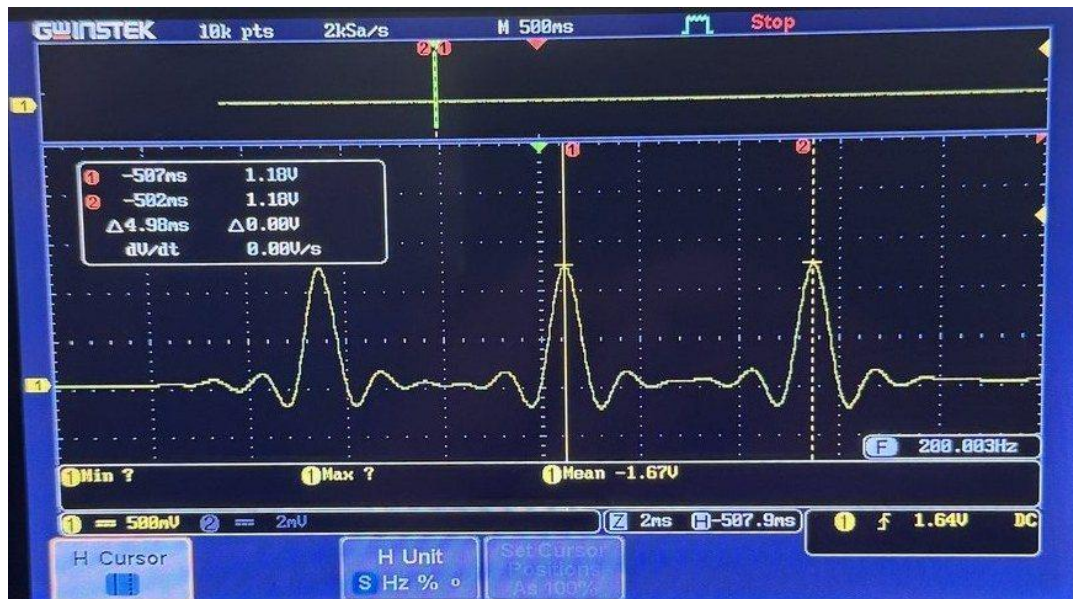


Figura 16. Trazo de ISR a 100 Hz con los cursores sobre dos pulsos consecutivos: $\Delta = 4.98 \text{ ms}$.

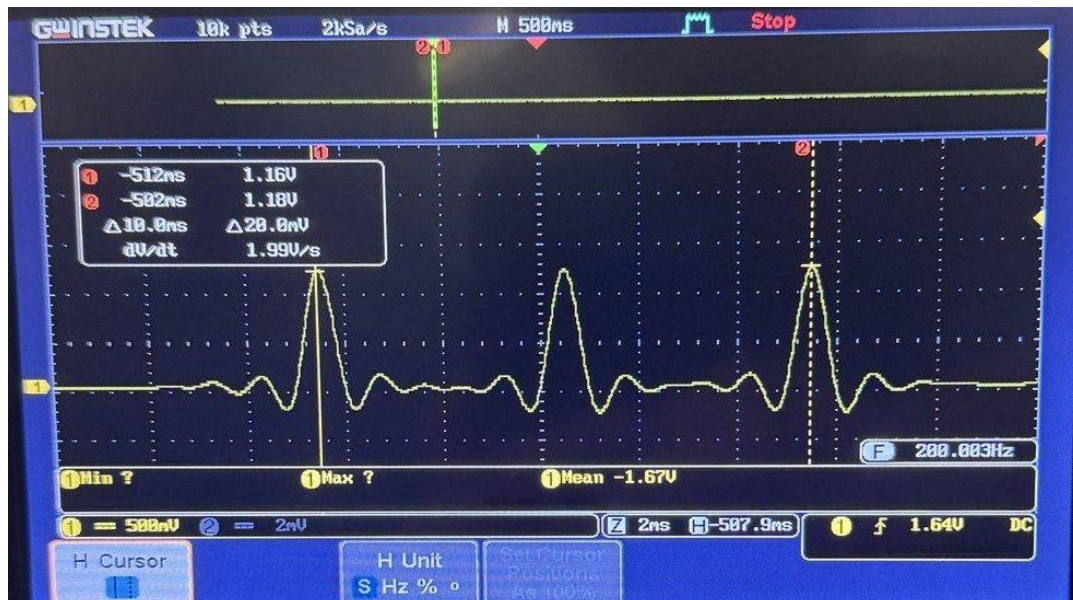


Figura 17. Misma captura con los cursores abarcando un periodo completo: $\Delta = 10.0 \text{ ms}$, equivalente a dos pulsos de traza.

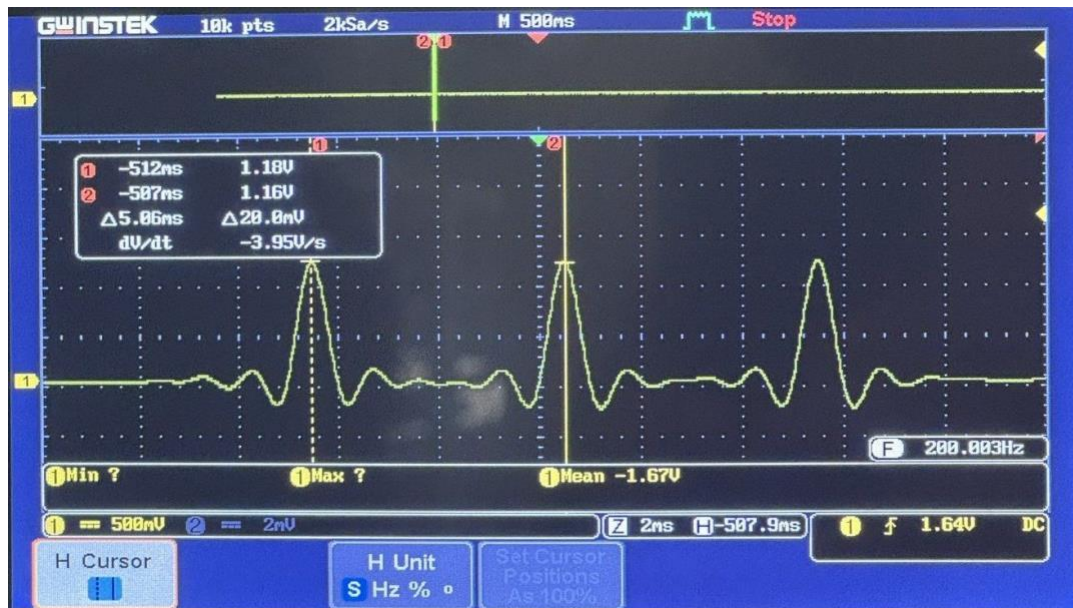


Figura 18. Vista ampliada de la traza a 100 Hz con $\Delta = 5.06 \text{ ms}$ entre pulsos.



Figura 19. Traza de ISR a 100 Hz con el contador de frecuencia en 200.003 Hz y $\Delta = 5.00 \text{ ms}$ entre pulsos.



Figura 20. Confirmación del intervalo de 5.00 ms en otro par de pulsos de la misma adquisición.



Figura 21. Tercera verificación del intervalo de 5.00 ms entre pulsos de traza.



Figura 22. Cursores abarcando tres intervalos consecutivos de traza: $\Delta = 15.0$ ms.

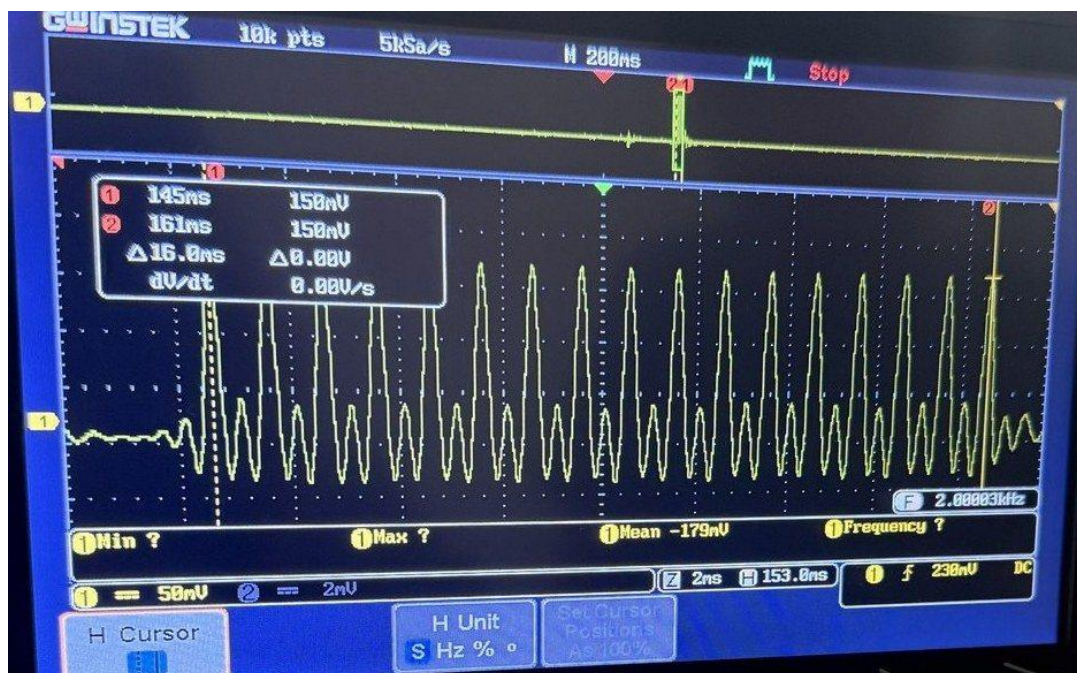


Figura 23. Trazo de ISR a 1 kHz con el contador de frecuencia en 2.00003 kHz y cursores en $\Delta = 16.0$ ms.



Figura 24. Detalle de la misma traza con $\Delta = 1.00\text{ ms}$, equivalente a un periodo de la señal de entrada.



Figura 25. Traza a 1 kHz con los cursores en $\Delta = 13.9\text{ ms}$ sobre el tren de pulsos.

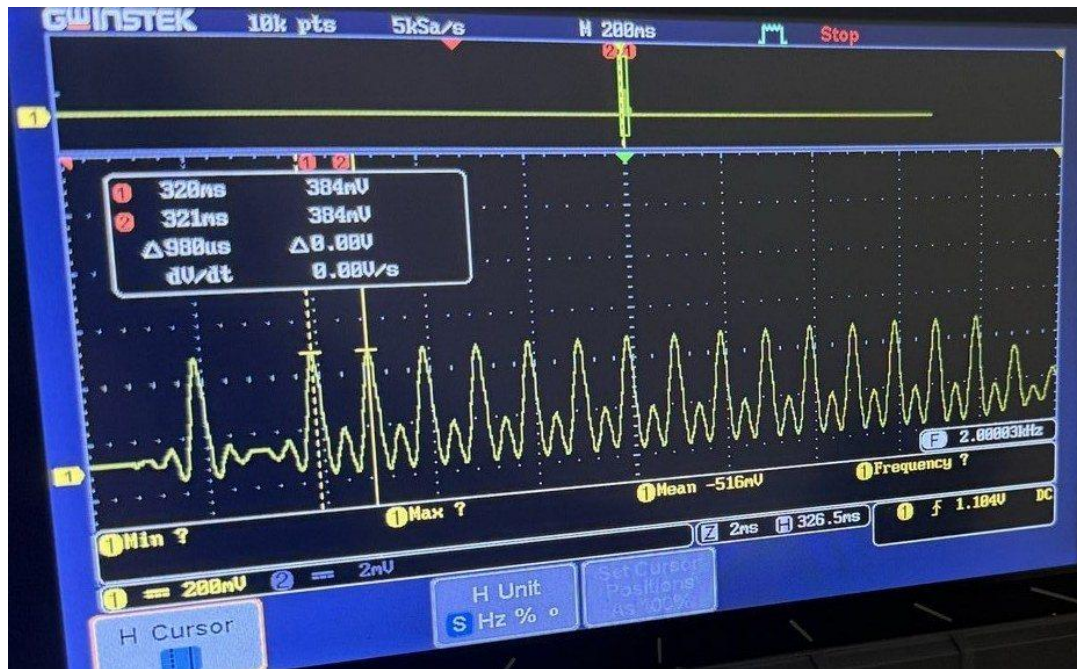


Figura 26. Detalle de la traza a 1 kHz con $\Delta = 980 \mu s$ entre dos pulsos separados.

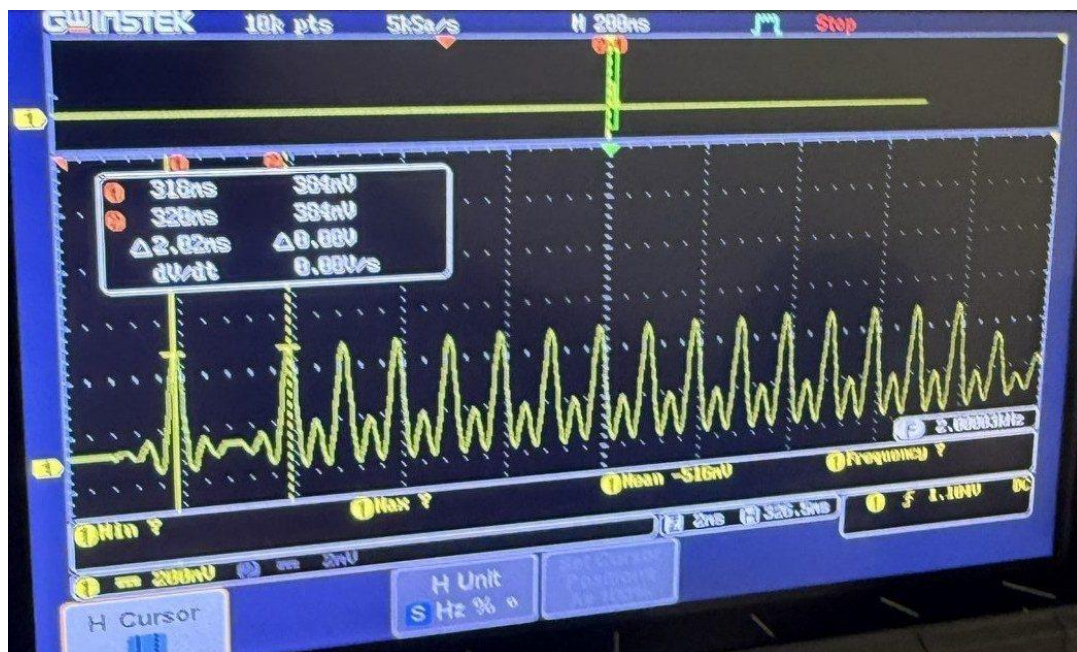


Figura 27. Detalle de la traza a 1 kHz con $\Delta = 2.02 ms$.



Figura 28. Traza de ISR a 10 kHz con el contador de frecuencia en 20.0003 kHz.

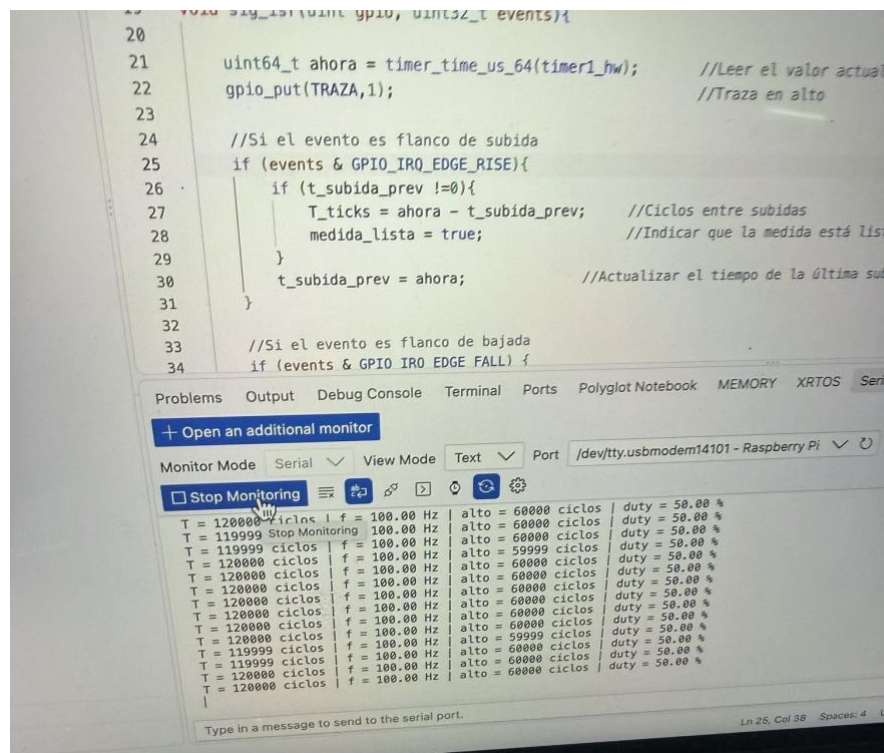


Figura 29. Medición en ciclos a 100 Hz: $T = 120000$ ciclos, $\text{alto} = 60000$ ciclos y $\text{duty} = 50.00\%$.



Figura 30. Generador configurado a 1 kHz para la corrida en modo ciclos.

```

20
21     uint64_t ahora = timer_time_us_64(timer1_hw);    //Leer el valor actual
22     gpio_put(TRAZA,1);                             //Traza en alto
23
24     //Si el evento es flanco de subida
25     if (events & GPIO_IRQ_EDGE_RISE){
26         if (t_subida_prev !=0){
27             T_ticks = ahora - t_subida_prev;        //Ciclos entre subidas
28             medida_lista = true;                   //Indicar que la medida está lista
29         }
30         t_subida_prev = ahora;                     //Actualizar el tiempo de la última su
31     }
32
33     //Si el evento es flanco de bajada
34     if (events & GPIO_IRQ_EDGE_FALL){

```

Problems Output Debug Console Terminal Ports Polyglot Notebook MEMORY XRTOS Se

+ Open an additional monitor

Monitor Mode Serial View Mode Text Port /dev/tty.usbmodem14101 - Raspberry Pi

Stop Monitoring

T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%
T = 12000	ciclos	f = 1000.00	Hz	alto = 6000	ciclos	duty = 50.00	%

Type in a message to send to the serial port.

Ln 25, Col 38

Figura 31. Medición en ciclos a 1 kHz: T = 12000 ciclos, alto = 6000 ciclos y duty = 50.00 %.

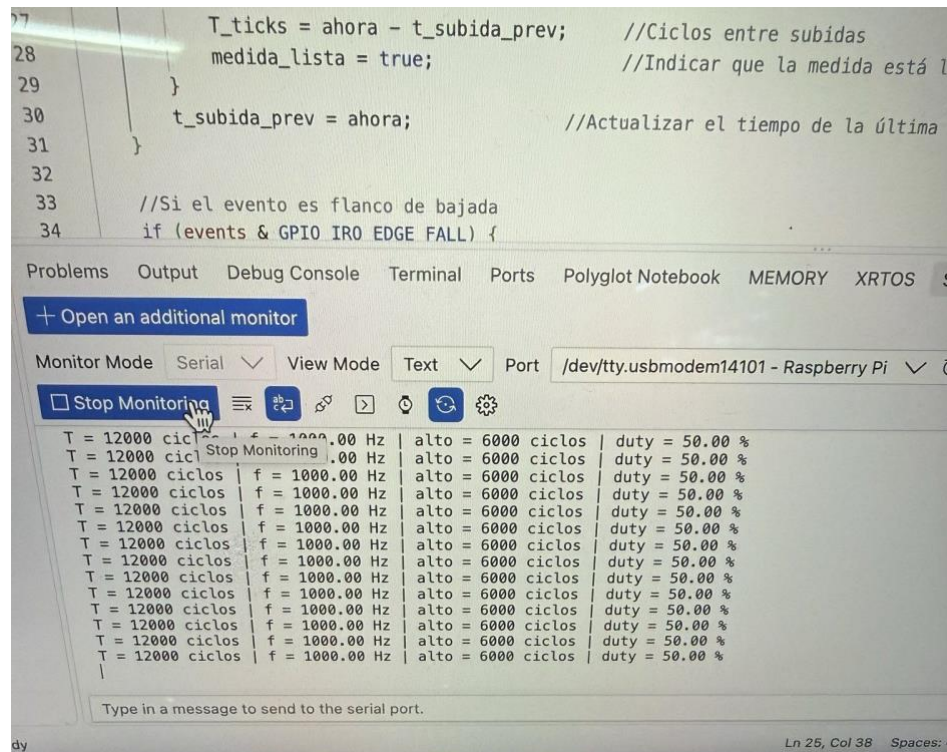


Figura 32. Corrida estable en modo ciclos a 1 kHz.



Figura 33. Generador configurado a 10 kHz para la corrida en modo ciclos.

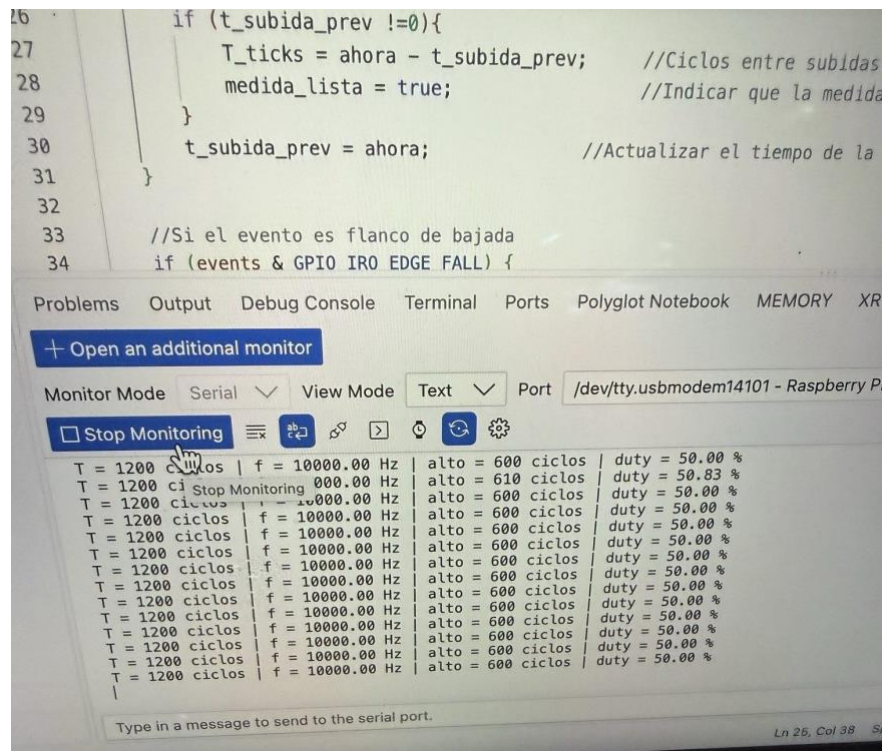


Figura 34. Medición en ciclos a 10 kHz: $T = 1200$ ciclos, $\text{alto} = 600$ ciclos y $\text{duty} = 50.00\%$.



Figura 35. Rebote del pulsador capturado con los cursores sobre la envolvente completa: $\Delta = 16.7$ ms.



Figura 36. Misma captura con los cursores ajustados al tramo de transiciones activas: $\Delta = 10.1$ ms.



Figura 37. Detalle de una transición individual dentro del rebote: $\Delta = 640$ μ s.

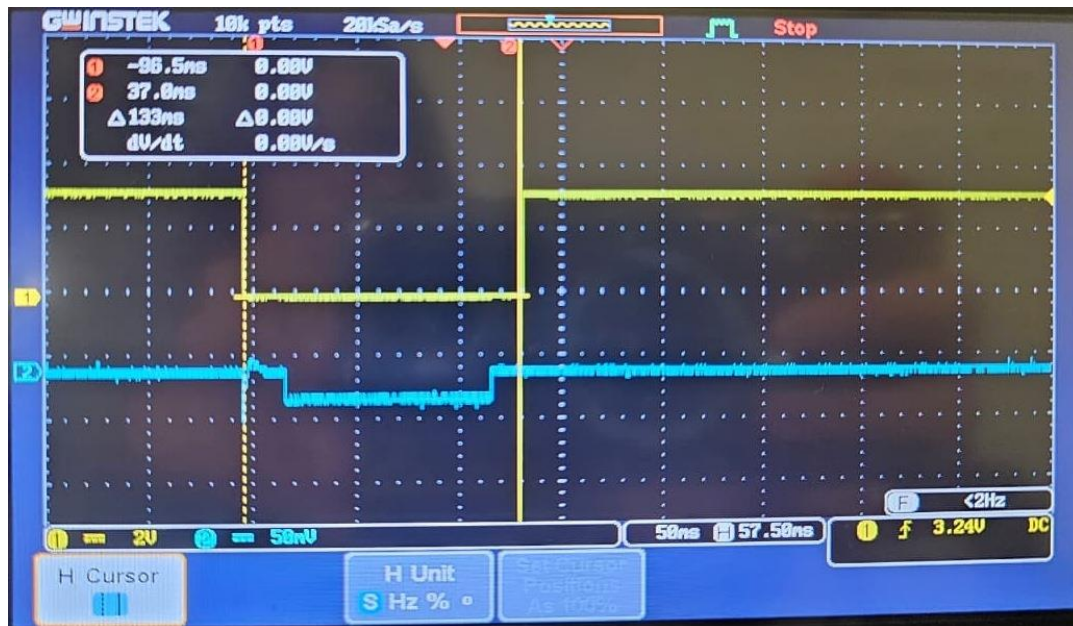


Figura 38. Captura de dos canales durante un accionamiento del pulsador, con los cursores abarcando $\Delta = 133\text{ ms}$.



Figura 39. Misma adquisición sin cursores, donde se aprecian el nivel del pulsador en el canal 1 y las transiciones del canal 2.

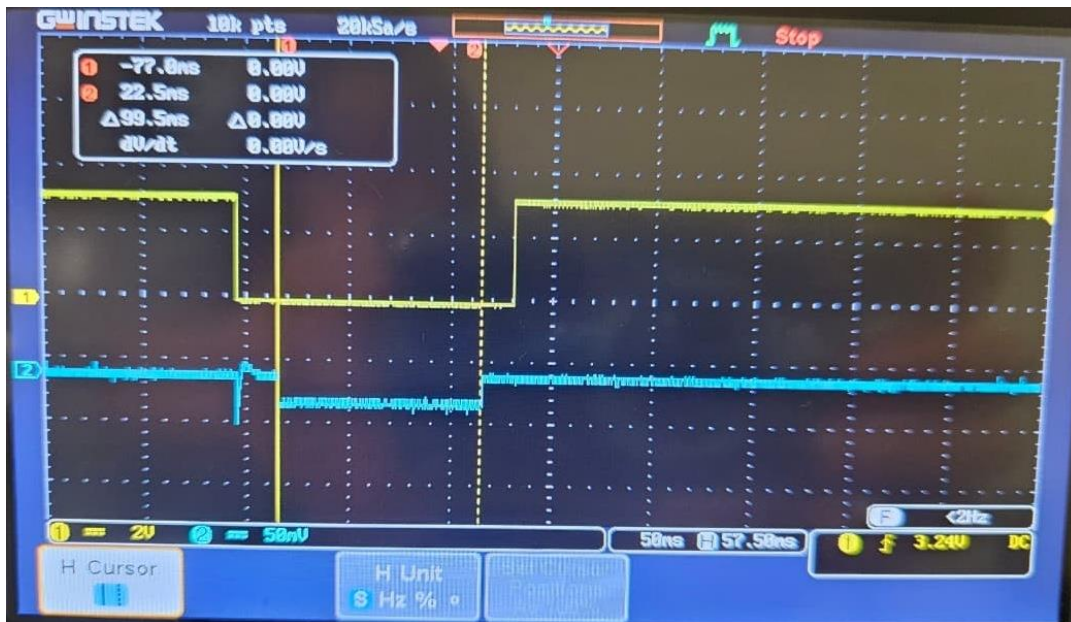


Figura 40. Segundo accionamiento del pulsador con los cursores sobre sus dos flancos: $\Delta = 99.5$ ms.

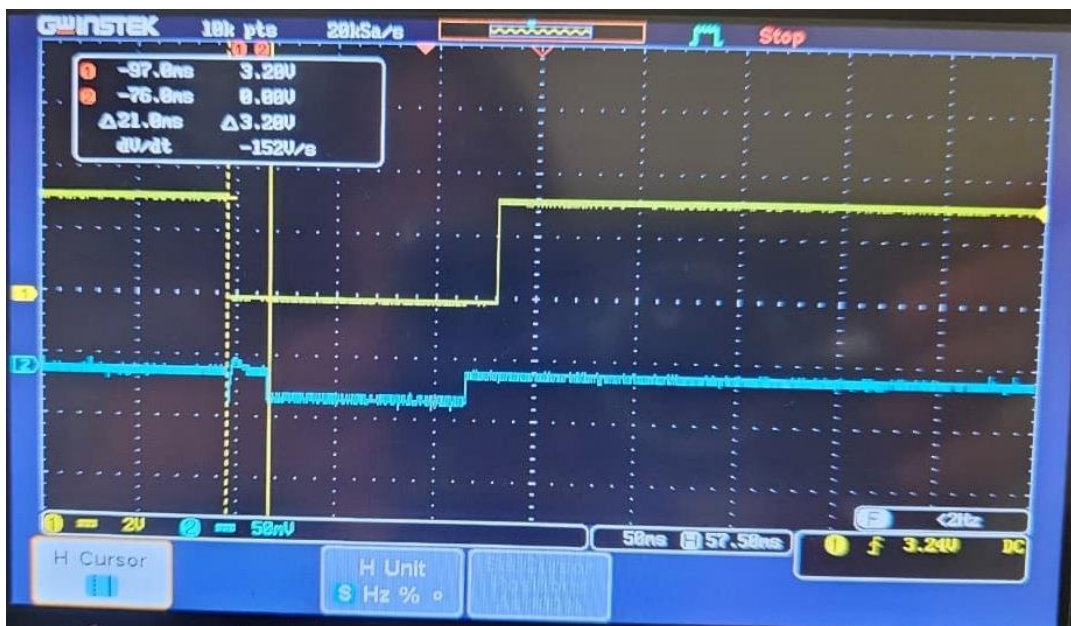


Figura 41. Intervalo entre el flanco del pulsador y la transición del canal 2: $\Delta = 21.0$ ms, que corresponde a la ventana de antirrebote programada.

Conclusiones

La práctica cumplió con el objetivo planteado al implementar y analizar dos técnicas complementarias de manejo del tiempo en el RP2350. El punto de partida fue la distinción entre generar y medir: aquí el instante lo impone un evento externo y, dado que el RP2350 carece de registro de captura, la copia del contador se resolvió por software, con la latencia de la interrupción como costo medible. Mantener dentro de la ISR únicamente la lectura de la marca de tiempo, y dejar la división y la impresión en el bucle

principal, fue lo que permitió medir a 10 kHz sin perder flancos, donde el intervalo entre transiciones consecutivas es de apenas 50 μ s. El cambio del tick de 1 μ s al de 83.33 ns mostró con claridad cómo la resolución gobierna la calidad de la medición: la fluctuación de un conteo que a 10 kHz desplazaba la lectura casi un uno por ciento desapareció por completo y la cota de error se redujo en el mismo factor de doce. Que la lectura aislada de 610 ciclos sobreviviera al cambio de modo, excediendo en cinco veces la cota de resolución, permitió atribuirle al jitter de la latencia y no al tick, lo que confirma que un presupuesto de error bien construido no solo acota el resultado sino que identifica la fuente de cada desviación. En la misma línea, la desviación constante de +15 ppm del contador de frecuencia se explica por la exactitud de las referencias de los instrumentos y no por el método, ya que un error de resolución sería constante en valor absoluto y creciente en términos relativos; de esa misma propiedad se desprende el techo de frecuencia, que bajo un criterio de uno por ciento de error se ubica alrededor de 5 kHz en modo microsegundos y cerca de 60 kHz en modo ciclos.

En la segunda parte el aporte principal fue medir el rebote en lugar de suponerlo: la envolvente de 16.7 ms obtenida con el osciloscopio es la evidencia que justifica la ventana de antirrebote de 20 ms y deja claro por qué una ventana menor a 10 ms confirmaría el estado del pin cuando el contacto todavía no se ha estabilizado. El esquema implementado descansa en una idea sencilla pero exigente, que la ISR no decide sino que únicamente agenda: tratar el primer flanco como un aviso, deshabilitar la interrupción del pin durante la ventana y confirmar por relectura en el callback es lo que convierte varias transiciones eléctricas en un solo evento lógico, sin bloquear la CPU ni exigir visitas periódicas desde el bucle principal. Conviene señalar las limitaciones del trabajo: las capturas de la traza se tomaron con bases de tiempo muy superiores a la duración del pulso, por lo que quedaron submuestreadas y solo resultan aprovechables a través del contador de frecuencia, de modo que no se determinó el tiempo de servicio de la ISR ni, en consecuencia, el segundo techo de frecuencia, aquel en que un flanco llega antes de que termine la rutina anterior; tampoco se realizó la determinación experimental del techo planteada como punto extra. Ambas mediciones son directas de obtener y quedan como continuación natural de esta práctica.

Referencias

1. Arm Limited. (2012). *Cortex-M0+ devices generic user guide* (DUI0662B) [Manual técnico]. <https://developer.arm.com/documentation/dui0662/b/>
2. Asparuhova, K., Shehova, D., Asenov, S., Kanevski, H., & Parushev, A. (2024). Using WOKWI simulator to support engineering student learning in microcontrollers and sensors. *2024 XXXIII International Scientific Conference Electronics (ET)* (pp. 1–4). IEEE. <https://doi.org/10.1109/ET63133.2024.10721553>

3. Belatik, A., Sabri, M. A., El Khoukhi, H., & Aarab, A. (2024). A comparative review of microcontroller architectures in embedded machine learning: The Raspberry Pi Pico (RP2040) and Arduino Nano 33 IoT (SAMD21). En S. Motahhir & B. Bossoufi (Eds.), *Digital technologies and applications (ICDTA 2024)* (Lecture Notes in Networks and Systems, Vol. 1100). Springer. https://doi.org/10.1007/978-3-031-68660-3_41
4. Raspberry Pi Ltd. (2024). *RP2040 datasheet: A microcontroller by Raspberry Pi* [Hoja de datos]. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>
5. Raspberry Pi Ltd. (2024). *RP2350 datasheet* [Hoja de datos]. <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
6. Raspberry Pi Ltd. (2024). *Raspberry Pi Pico-series C/C++ SDK* [Manual técnico]. <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>
7. Skarman, F., Klemmer, L., Große, D., Gustafsson, O., & Laeuffer, K. (2025). Surfer — An extensible waveform viewer. En R. Piskac & Z. Rakamarić (Eds.), *Computer aided verification (CAV 2025)* (Lecture Notes in Computer Science, Vol. 15934, pp. 392–404). Springer. https://doi.org/10.1007/978-3-031-98685-7_19

Declaración de IA

- a) **Claude:** Se usó para referencias de marco teórico y organización del PDF.
- b) **ChatGPT:** Se usó para código y comentado.